Heap Sort

Heap sort is a comparision-based sorting algorithm that uses a binary heap data structure to sort elements. It works by first building a max heap (for ascending order), then repeatedly swapping the largest element (root of the heap) with the last element of the array, reducing the heap size, and ~~heapifying~~ heapifying the root again.

Steps:-

1.> Build a max heap from the input data.

2.> Swap the root (maximum value) with the last item of the heap.

3.> Reduce the size of the heap by one and heapify the root.

4.> Repeat step 2 and 3 until the heap is empty.

Akash

→ // c program for heap sort;

```
#include<stdio.h>

// function to heapify a subtree rooted at index i
void heapify (int arr[], int n, int i)
{
    int largest = i; // initialize largest as root
    int left = 2*i + 1; // left child index
    int right = 2*i + 2; // right child index

// IF left child is larger than root
if (left<n && arr[left] > arr [largest])
{
    largest = left;
}

// IF right child is larger than largest so far
    if ( right < n && arr[right] > arr[largest])
{
    largest = right;
}

// IF largest is not root
if (largest != i)
{
                                    // swap (arr[i], arr[largest])
    int temp = arr [i];
    arr[i] = arr [largest];
    arr[largest] = temp;
```

Akash

```
// recursively heapify the affected sub-tree
    heapify (arr, n, largest);
  }
}


// main function to do heap sort
void heapsort (int arr[], int n)
  {
      // build max heap
      int = i;
    for (i= n/2 -1 ; i>=0; i--)
      {
          heapify (arr, n, i);
      }
    // one by one extract elements from heap
    for (i=n-1; i>0; i--)
      {                                  //swap (arr[0], arr[i])
          int temp = arr[0];
              arr[0] = arr[i];
              arr[i] = temp;

      heapify (arr, i, 0);
    }
}


int main()
  {
      int a[20], n, i;
      printf("enter size of an array");
      scanf("%d", &n);
```

```
for(i=0; i<n; i++)
  {
    printf("enter data \n");
    scanf("%d", &a[i]);
  }

heapsort (arr, n);

printf("sorted array using heap sort: \n");

for(i=0; i<n; i++)
  {
    printf("%d ", a[i]);
  }
return 0;
}
```

Akash

→ Max heap →

~~① The largest element is always at the root~~

~~② keep removing the~~

① It must be a binary tree (each node has at most 2 children).

② Complete tree → All levels are completely filled except possibly the last, which is filled from left to right

③ Max-heap property
for every node i

- value(i) ≥ value(left child)
- value(i) ≥ value(right child)

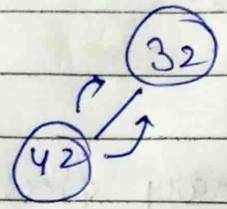
Akash

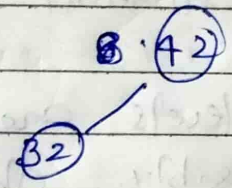
Q. Describe the insertion in max heap tree with example from the following list of numbers: 32, 42, 67, 23, 44, 49, 74.

=>

Step 1 → 32 add, 32 is root

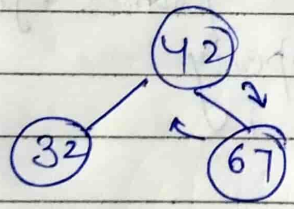
2 → add 42; 32 < 42, swap

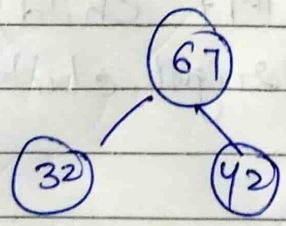




add 67;

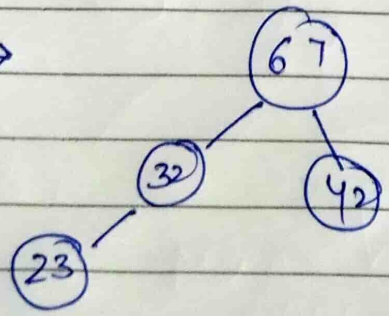
3 → 42 < 67, swap





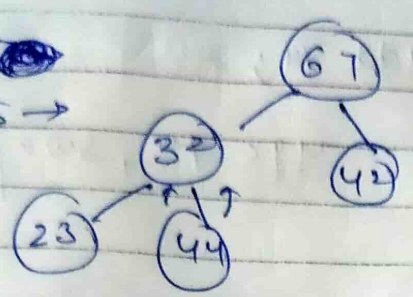
add 23;

4 → 32 > 23, OK



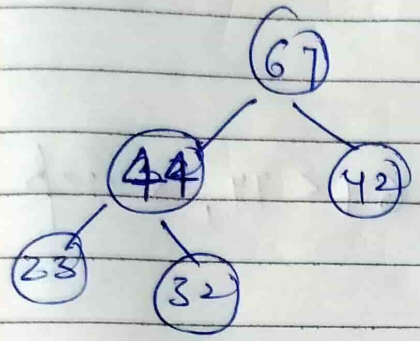
Akash

5 →

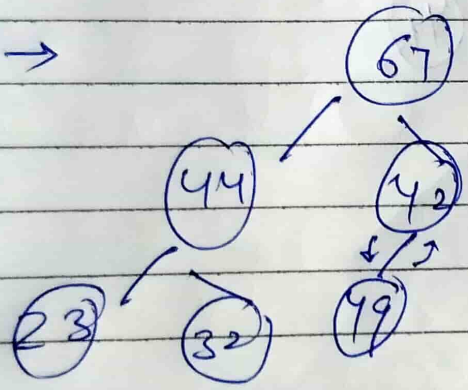


add 44;

32 < 44, swap

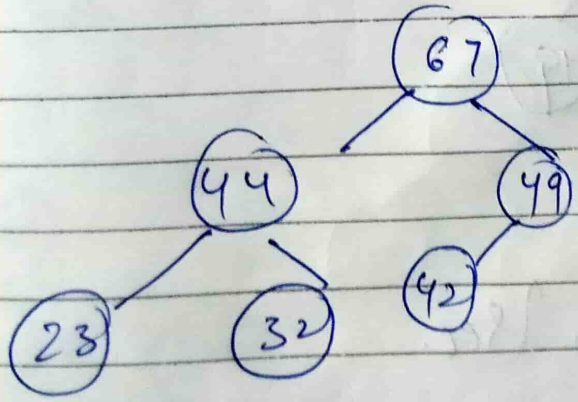


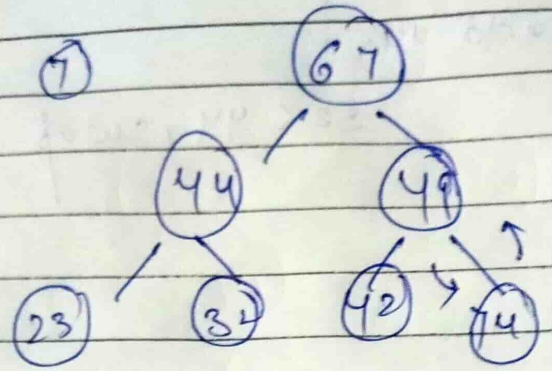
6 →



add 49;

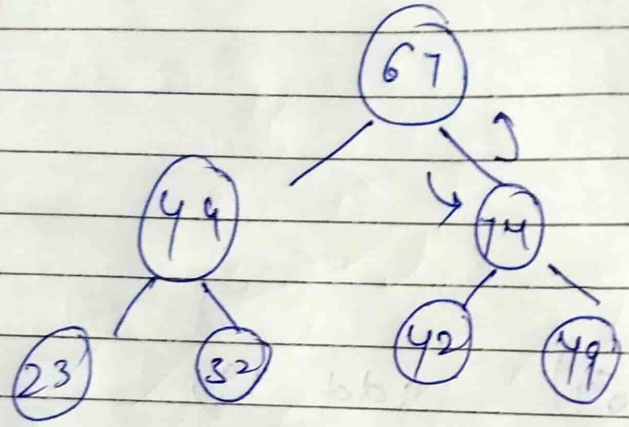
42 < 49, swap



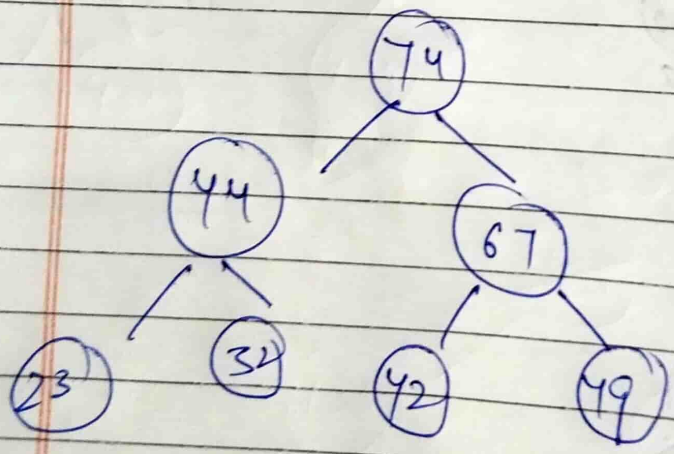


add 74;

49 < 74, swap



67 < 74, swap



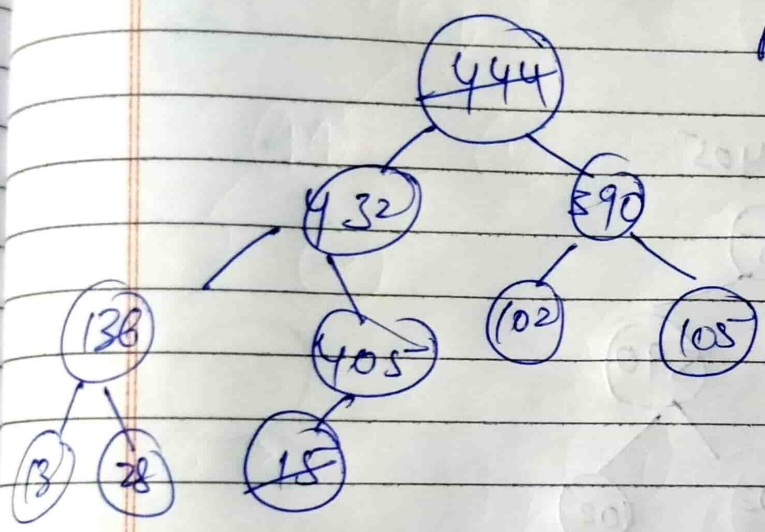
Max-heap Tree

Akash

→ Eg; for heap sort;

13, 102, 405, 136, 15, 105, 390, 432, 28, 444.

⇒ Max-heap tree :-

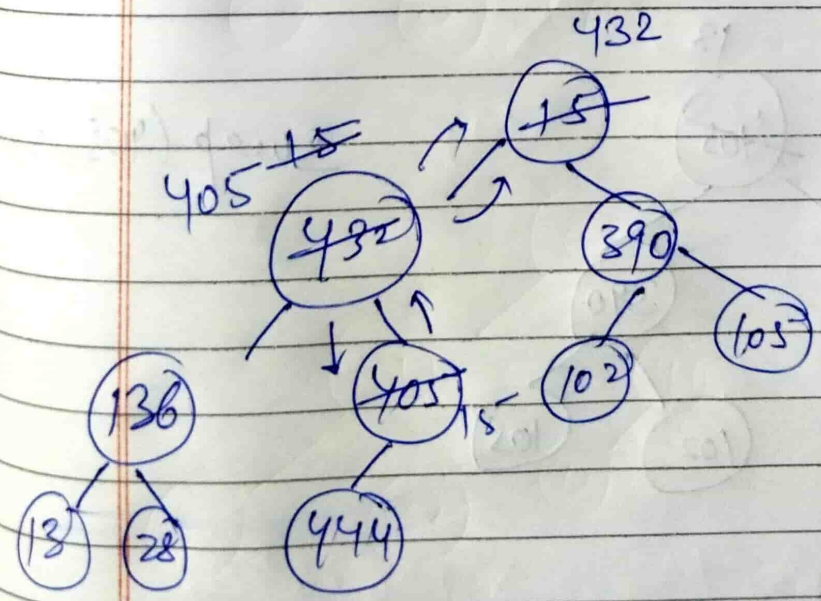


Now, heapifying

Swap (a[last], a[0])

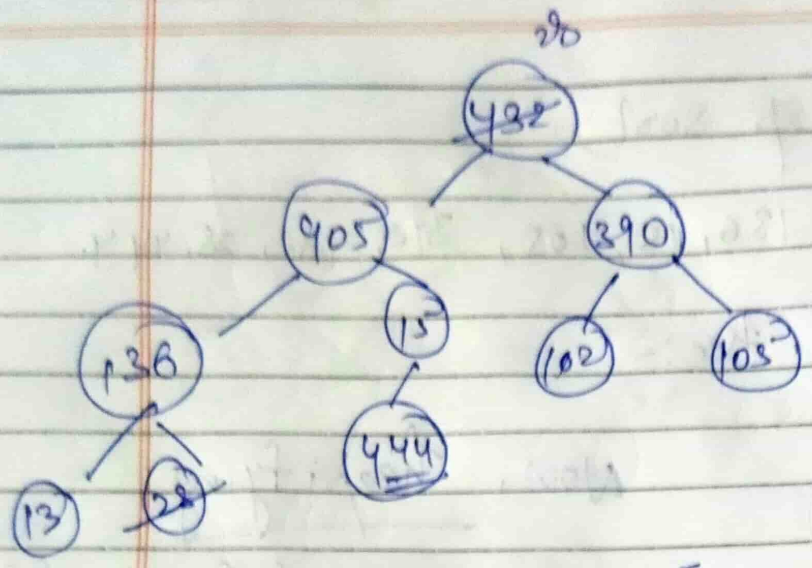
Swap (444, 15)

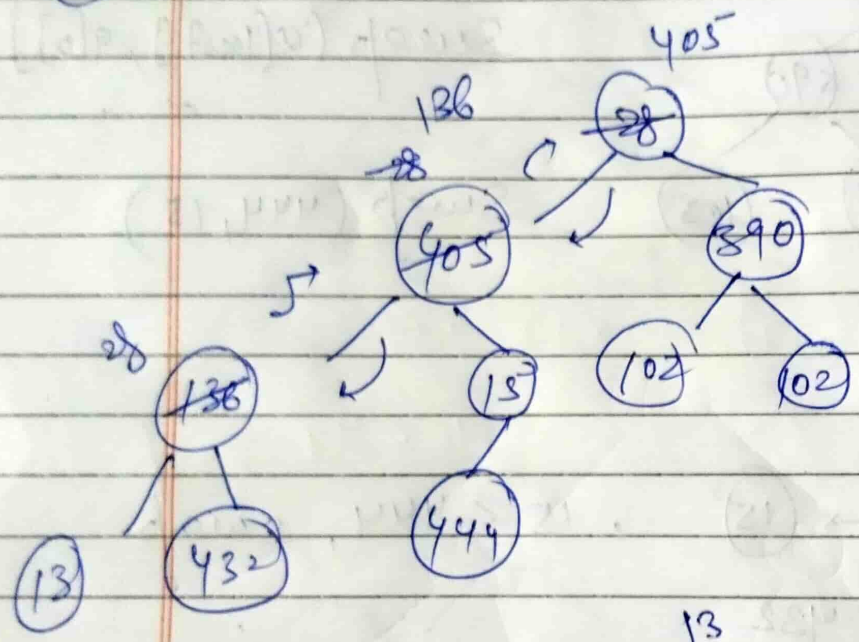
last one → 15 , 15 < 444, swap



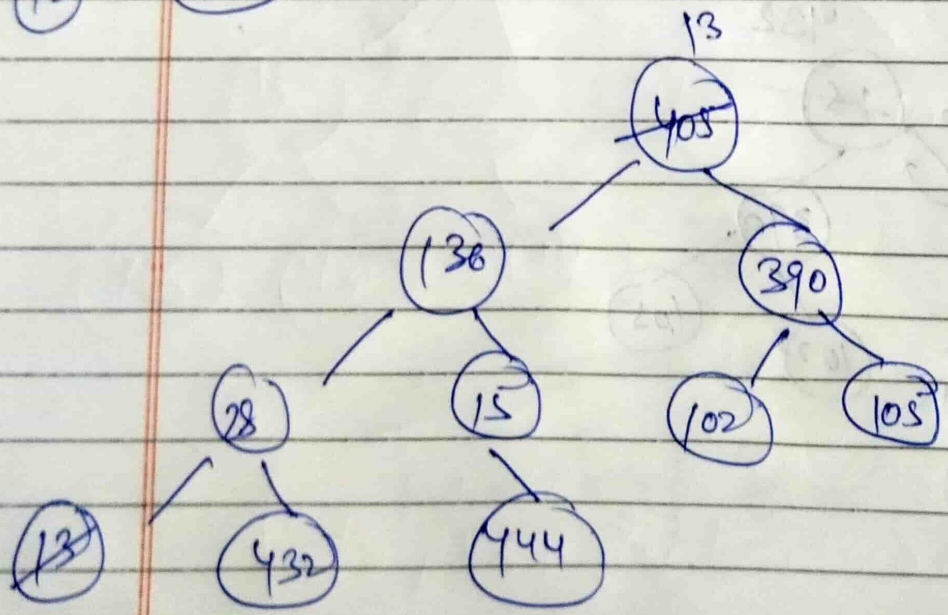
Akash

swap (28, 432)

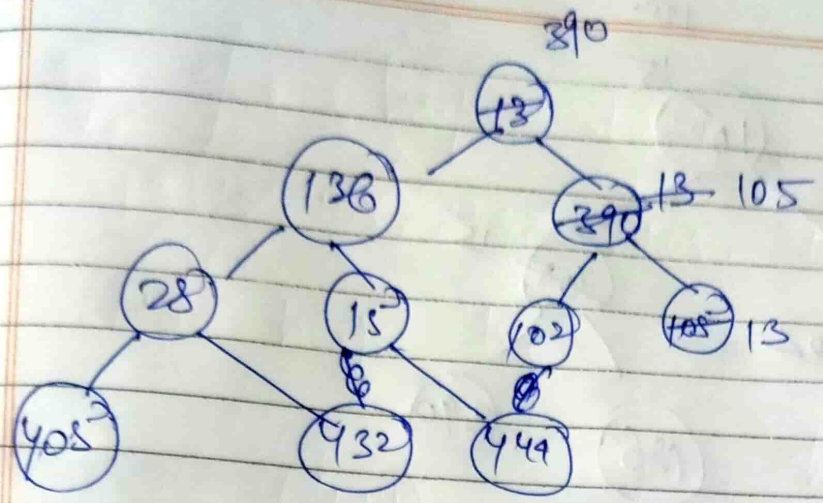


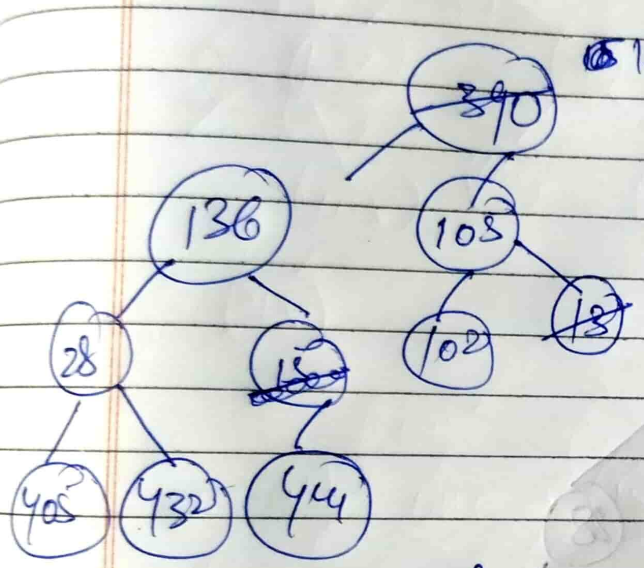


swap (405, 13)

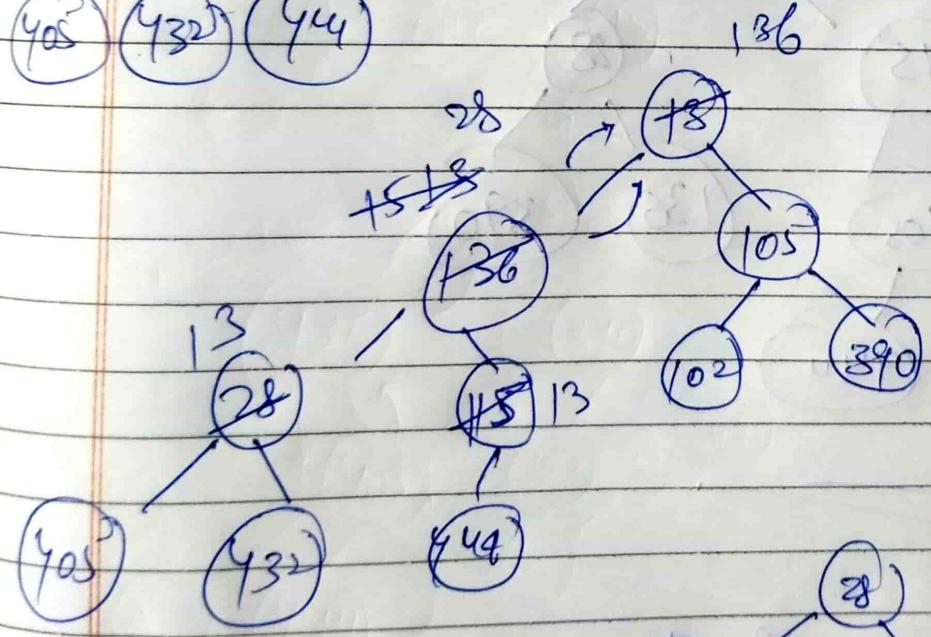


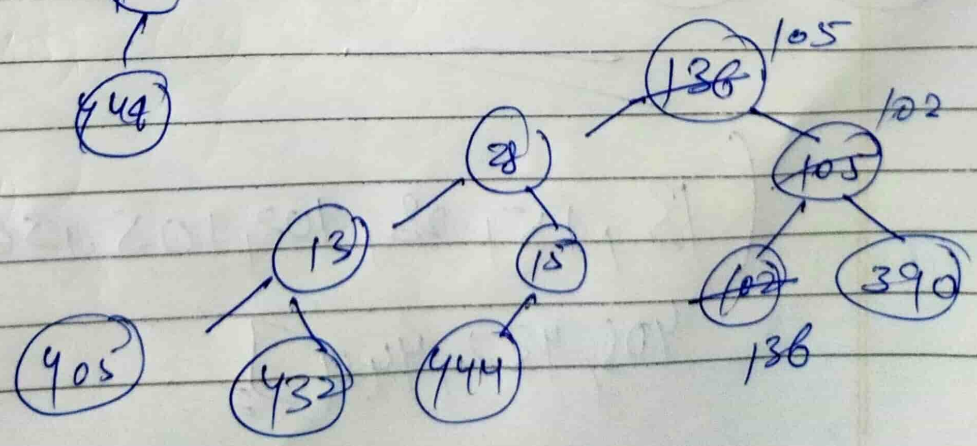
Akash

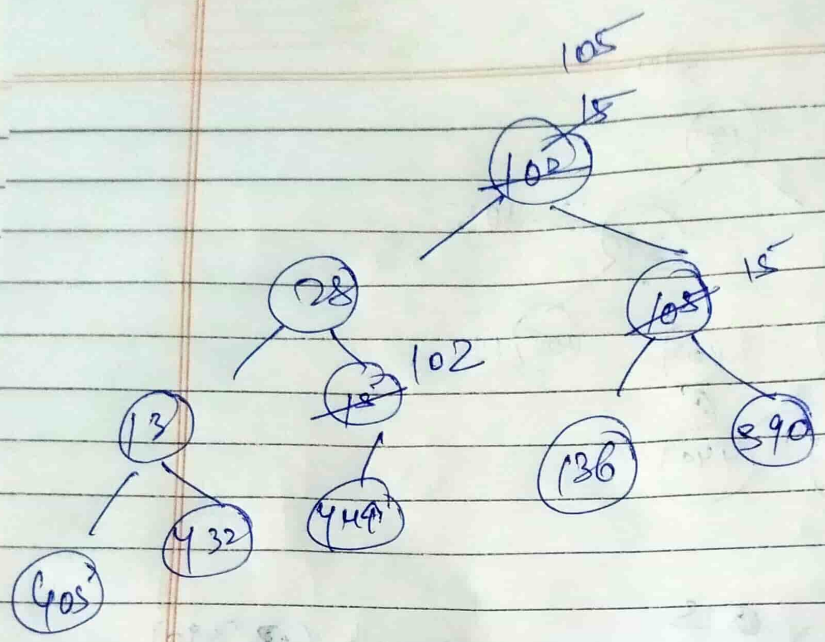

390
13
136
390
13
105
28
15
102
105
13
405
432
444


13
390
136
105
28
15
102
13
405
432
444

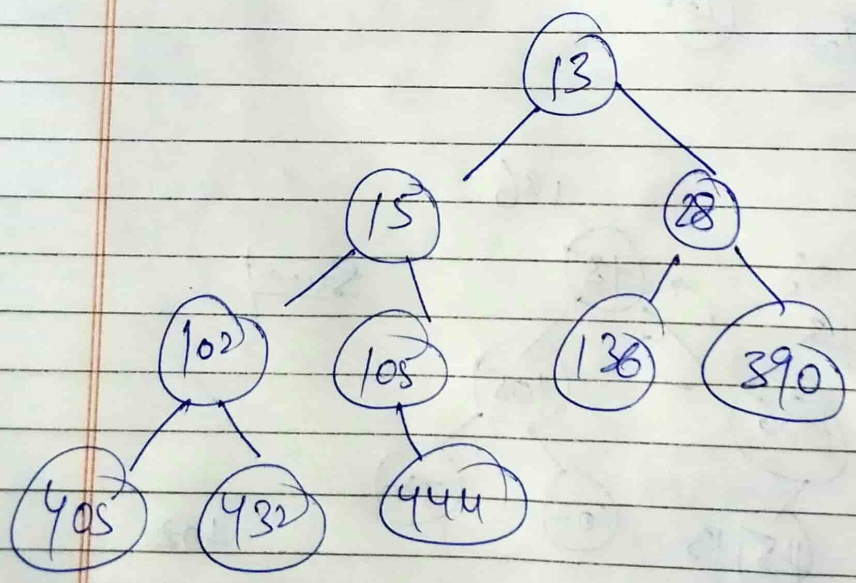
swap (13, 390)


136
28
13
105
13
136
102
390
13
28
15
13
405
432
444


102
105
136
102
28
105
13
15
102
390
136
405
432
444



Similar



[13, 15, 28, 102, 105, 136, 390, 405, 432, 444]

Time Complexity

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Akash

Akash